

Lab 4: Task-Priority kinematic control (1B)



HANDS-ON INTERVENTION: *Vehicle-Manipulator Systems*

Report by: PRAVIN OLI (u1999699), IFROS
(E-mail: pravin.oli.08@gmail.com)

Code Group : Pravin Oli and Gebreslassie Gebrecherkos Gebrezgabhier

Abstract—This lab explores the implementation of Task-Priority (TP) control for a 3-link planar manipulator, divided into two exercises. In Exercise 1, the recursive Task-Priority algorithm is implemented, enabling hierarchical task execution for the manipulator, with tasks such as position control and configuration control. The tasks are defined as Python classes and integrated into the control framework, including visualizing the robot's motion and error evolution. Exercise 2 extends the implementation to provide flexible task definitions, including link selection for position and orientation control, incorporation of gain matrices for Damped Least Squares (DLS) inverse kinematics, and feedforward velocity for tracking dynamic trajectories.



1 INTRODUCTION

THIS lab explores the implementation of **task-priority control** for a **3-link planar robotic manipulator**. The lab is divided into two exercises, each focusing on different aspects of motion control and task prioritization.

In **Exercise 1**, we implement the **Task-Priority (TP) algorithm** in its **recursive form**, enabling an **arbitrary hierarchy of tasks** for the manipulator. This involves defining different robotic tasks as Python classes and integrating them into a recursive control framework. The simulation includes:

- Defining **Denavit-Hartenberg (DH) parameters** for the **3-link planar manipulator**.
- Implementing **task-priority control** using different task hierarchies.
- Visualizing the robot's motion and tracking **task error evolution** over time.

In **Exercise 2**, the implementation is extended to provide more **flexible task definitions**. This includes:

- **Selecting specific links** for **position and orientation** control.
- Introducing **gain matrices** for **weighted Damped Least Squares (DLS) inverse kinematics**.
- Implementing **feedforward velocity components** to allow the robot to track dynamic trajectories.

The **Task-Priority algorithm** enables the manipulator to execute multiple tasks simultaneously, prioritizing them in a **hierarchical** manner.

1.1 Objectives

The objective of this lab are:

- *Erasmus Mundus Joint Masters In Intelligent Field Robotic Systems, University of Girona, Spain.*

- To implement the Task-Priority algorithm in its recursive form for a 3-link planar manipulator, enabling an arbitrary hierarchy of tasks.
- To extend the implementation by incorporating flexible task definitions, including link selection for position and orientation tasks, gain matrices with weighted Damped Least-Squares (DLS) implementation, and feedforward velocity for tracking.

2 PLANER ROBOTIC MANIPULATOR

A planar manipulator operates in a two-dimensional plane, typically with revolute or prismatic joints.

2.1 Planar Manipulator with 3-link and 3 revolute joints.

A planar 3-link revolute manipulator consists of three rotational joints that operate in the XY-plane. The configuration of the manipulator is described by the joint angles θ_1 , θ_2 , and θ_3 .

$$\text{State} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

where:

- θ_1 is the angle of the first revolute joint.
- θ_2 is the angle of the second revolute joint.
- θ_3 is the angle of the third revolute joint.

$$T_n^{n-1} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

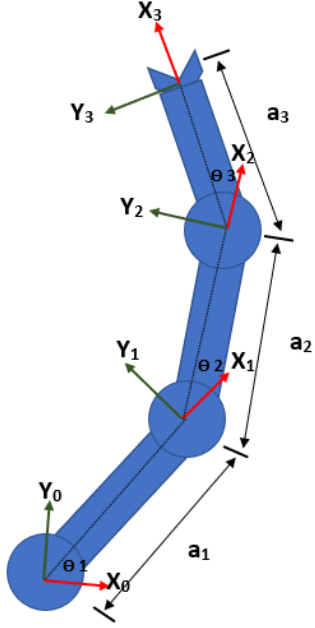


Fig. 1: 3 joint planer manipulator

i	a_i (m)	α_i (rad)	d_i (m)	θ_i (rad)
1	0.8	0	0	$q_1 = \frac{\pi}{12}$
2	0.6	0	0	$q_2 = \frac{\pi}{6}$
3	0.4	0	0	$q_3 = \frac{\pi}{3}$

TABLE 1: Updated Denavit-Hartenberg Parameters based on the implemented code

2.2 Inequality Tasks Concept

Inequality tasks are used to manage constraints in robot control systems, particularly when the robot's motion must remain within specific limits, such as joint position limits or safety zones. These tasks ensure that a robot's task variables are maintained within safe or desired regions. They are especially relevant in systems where safety is a priority, and they often appear at the top of the task hierarchy.

2.3 Equality Task Concept

An equality task defines a task variable $\sigma_i(q)$ and its associated task definition:

$$\dot{x}_i = \dot{\sigma}_i + K\tilde{\sigma}_i$$

Where $\tilde{\sigma}_i = \sigma_{i,d} - \sigma_i$, representing the error in the task variable, and J_i is the Jacobian associated with the task.

2.4 Inequality Task (Set Task)

The inequality task focuses on keeping the task variable within a specific set of values, defined by a safe set:

$$\sigma_i(q) \in \mathbb{R}^{m_i}, \quad \mathcal{S}_i = [q_{i,min}, q_{i,max}]$$

This safe set ensures that the task variable stays within the desired range. These tasks typically relate to safety and are placed at the top of the task hierarchy, allowing other tasks to be subordinate. A switching mechanism is implemented to activate the task only when the robot is approaching the desired state (e.g., a target position).

2.5 Joint Limits Task

A specific type of inequality task is the **joint limits task**, where the task is to keep the joint angles within safe limits. This task ensures that the robot does not reach beyond its physical joint limits, which could result in damage or unsafe configurations.

The task switching is defined by activation and deactivation thresholds α_{li} and δ_{li} , respectively, which determine when the task should be activated or deactivated:

$$a_{li}(q) = \begin{cases} -1, & \text{if } q_i \geq q_{i,max} - \alpha_{li} \\ 1, & \text{if } q_i \leq q_{i,min} + \alpha_{li} \\ 0, & \text{if } q_{i,min} < q_i < q_{i,max} \end{cases}$$

This switching function helps avoid oscillations (chatter) when the joint positions are close to the limits.

2.6 Extension of the Recursive Task Priority (TP)

Recursive Task Priority (TP) is an extension of the standard Task Priority method that efficiently handles multiple tasks in robotic systems.

- **Higher-priority tasks** are executed first.
- **Lower-priority tasks** are executed in the null space of the higher-priority ones, meaning they only influence the robot's motion when there are no constraints from higher-priority tasks.

The recursive TP algorithm solves for the robot's quasi-velocities ζ_k while considering multiple tasks in a priority hierarchy. The general steps for the algorithm are as follows:

2.6.1 Initialization

Initializing the quasi-velocity for the first task as zero:

$$\zeta_0 = 0^n$$

The initial projection matrix P_0 is the identity matrix:

$$P_0 = I^{n \times n}$$

2.6.2 Task Iteration

For each task i in the list of tasks $i = 1 \dots k$, where k is the number of tasks and n is the number of degrees of freedom (DOF) of the robot.

2.6.3 Task Activation

If the activation function $a_i(q)$ is non-zero (meaning the task is active):

$$a_i(q) \neq 0$$

Then, update the task's Jacobian $J_i(q)$ and compute the quasi-velocity ζ_i using the recursive relationship:

$$\zeta_i = \zeta_{i-1} + J_i(q)a_i(q)\dot{x}_i - J_i(q)P_{i-1}\zeta_{i-1}$$

where:

- $J_i(q)$ is the Jacobian of task i ,
- $a_i(q)$ is the activation function,
- $\dot{x}_i$ is the desired velocity for task i ,
- P_{i-1} is the projection matrix from the previous task.

Update the projection matrix P_i as:

$$P_i = P_{i-1} - J_i^T(q)J_i(q)$$

2.6.4 Task Inactivity

If the task is not active (i.e., $a_i(q) = 0$), the quasi-velocity and projection matrix do not change:

$$\zeta_i = \zeta_{i-1}, \quad P_i = P_{i-1}$$

2.6.5 Final Quasi-Velocity

After iterating through all tasks, the final quasi-velocity is returned:

$$\zeta_k$$

Algorithm 1 Recursive Task Priority Control Algorithm

Require: List of tasks $\{J_i(q), \dot{x}_i(q), a_i(q)\}$, where $i \in \{1, \dots, k\}$

Ensure: Quasi-velocities $\zeta_k \in \mathbb{R}^n$

```

1: Initialize:  $\zeta_0 \leftarrow 0^n$ ,  $P_0 \leftarrow I^{n \times n}$ 
2: for  $i = 1$  to  $k$  do
3:   if  $a_i(q) \neq 0$  then
4:     Compute Jacobian projection:  $\bar{J}_i(q) \leftarrow J_i(q)P_{i-1}$ 
5:     Update quasi-velocity:
        $\zeta_i \leftarrow \zeta_{i-1} + \bar{J}_i^\dagger(q) (a_i(q)\dot{x}_i(q) - J_i(q)\zeta_{i-1})$ 
6:     Update projection matrix:
        $P_i \leftarrow P_{i-1} - \bar{J}_i^\dagger(q)J_i(q)$ 
7:   else
8:      $\zeta_i \leftarrow \zeta_{i-1}$ 
9:      $P_i \leftarrow P_{i-1}$ 
10:  end if
11: end for
12: return  $\zeta_k$ 

```

3 IMPLEMENTATION

This lab is implemented in three Python scripts: **lab2 robotics.py** (the same as the previous lab), **lab4 robotics.py** and **lab4 exercise12.py**

3.1 lab2 robotics.py

This is the same as in the previous lab. I have added pseudo-inverse calculation.

```

def PseudoInverseJacob(J):
    return np.linalg.pinv(J)

```

3.2 lab4 robotics.py

Below is an implementation of lab4 robotics.py:

```

1 from lab2_robotics import * # Includes numpy import
2
3 def jacobianLink(T, revolute, link): # Needed in
4   Exercise 2
5   '''
6   Function builds a Jacobian for the end-
7   effector of a robot,
8   described by a list of kinematic
9   transformations and a list of joint
10  types.
11
12  Arguments:
13  T (list of Numpy array): list of
14  transformations along the kinematic
15  chain of the robot (from the base frame)

```

```

    revolute (list of Bool): list of flags
        specifying if the corresponding joint is
        a revolute joint
    link(integer): index of the link for which
        the Jacobian is computed

    Returns:
    (Numpy array): end-effector Jacobian
'''
# Code almost identical to the one from
lab2_robotics...

J = np.zeros((6, len(T)-1)) # Empty Jacobian (6
x N) where N - number of joints
O = T[-1][:3, 3] # End-effector position

for i in range(link):
    # a. Extract z and o.
    z = T[i][0:3, 2]
    o = T[i][0:3, 3]

    # b. Check joint type.
    if revolute[i]:
        J1_3 = np.cross(z, (O - o))
        J[:,i] = np.concatenate((J1_3, z), axis
=0)
    else:
        J[:,i] = np.concatenate((z, np.zeros(3))
, axis=0)

return J
'''
''' Class representing a robotic manipulator.
'''
class Manipulator:
    '''
    Constructor.

    Arguments:
    d (Numpy array): list of displacements along
Z-axis
    theta (Numpy array): list of rotations
around Z-axis
    a (Numpy array): list of displacements along
X-axis
    alpha (Numpy array): list of rotations
around X-axis
    revolute (list of Bool): list of flags
specifying if the corresponding joint is
a revolute joint
'''
def __init__(self, d, theta, a, alpha, revolute)
:
    self.d = d
    self.theta = theta
    self.a = a
    self.alpha = alpha
    self.revolute = revolute
    self.dof = len(self.revolute) # Number of
degrees of freedom
    self.q = np.zeros(self.dof).reshape(-1, 1)
    # Joint variables initialized to zero
    self.update(0.0, 0.0) # Initialize
transformations

'''
    Method that updates the state of the robot.

    Arguments:
    dq (Numpy array): a column vector of joint
velocities
    dt (double): sampling time
'''
def update(self, dq, dt):
    self.q += dq * dt # Integrates joint
velocities over time step

```

```

68     for i in range(len(self.revolute)):
69         if self.revolute[i]:
70             self.theta[i] = self.q[i] # Updates
71             revolute joints
72         else:
73             self.d[i] = self.q[i] # Updates
74             prismatic joints
75     self.T = kinematics(self.d, self.theta, self
76     .a, self.alpha) # Recompute forward
77     kinematics
78
79     '''
80     Method that returns the characteristic
81     points of the robot.
82     '''
83     def drawing(self):
84         return robotPoints2D(self.T)
85
86     '''
87     Method that returns the end-effector
88     Jacobian.
89     '''
90     def getEEJacobian(self):
91         return jacobian(self.T, self.revolute)[:2,
92         :2]
93
94     '''
95     Method that returns the orientation jacobian
96     Jacobian.
97     '''
98     def getOrientationJacobian(self, link):
99         return self.getLinkJacobian(link)[5, :] #
100         Orientation Jacobian
101
102     '''
103     Method that returns the configuration
104     Jacobian.
105     '''
106     def getConfigJacobian(self, link):
107         J = np.block([[self.getLinkJacobian(link)
108         [0:2, :].reshape(2,3)],
109         [self.getOrientationJacobian(
110         link).reshape(1,3)]]])
111         return J
112
113     '''
114     Method that returns the end-effector
115     transformation.
116     '''
117     def getEETransform(self):
118         return self.T[-1]
119
120     '''
121     Method that returns the position of a
122     selected joint.
123
124     Argument:
125     joint (integer): index of the joint
126
127     Returns:
128     (double): position of the joint
129     '''
130     def getJointPos(self, joint):
131         return self.q[joint]
132
133     '''
134     Method that returns the transformation of a
135     selected link.
136
137     Argument:
138     link (integer): index of the link
139
140     Returns:
141     (Numpy array): transformation of the link'''
142     def getLinkTransform(self, link):
143         return self.T[link]
144
145     '''
146     Method that returns the Jacobian of a
147     selected link.
148
149     Argument:
150     link (integer): index of the link
151
152     Returns:
153     (Numpy array): Jacobian of the link'''
154     def getLinkJacobian(self, link):
155         return jacobianLink(self.T, self.revolute,
156         link)
157
158     '''
159     Method that returns number of DOF of the
160     manipulator.
161     '''
162     def getDOF(self):
163         return self.dof
164
165     '''
166     Base class representing an abstract Task.
167     '''
168     class Task:
169         '''
170         Constructor.
171
172         Arguments:
173         name (string): title of the task
174         desired (Numpy array): desired sigma (goal)
175         '''
176         def __init__(self, name, desired, link, v_ff,
177         K):
178             self.name = name # task title
179             self.sigma_d = desired # desired sigma
180             self.v_ff = v_ff # feedforward velocity
181             self.K = K # gain matrix
182             self.link = link
183
184         '''
185         Method updating the task variables (abstract
186         ).
187
188         Arguments:
189         robot (object of class Manipulator):
190         reference to the manipulator
191         '''
192         def getNmae(self):
193             return self.name
194
195         def update(self, robot):
196             pass
197
198         '''
199         Method setting the desired sigma.
200
201         Arguments:
202         value(Numpy array): value of the desired
203         sigma (goal)
204         '''
205         def setDesired(self, value):
206             self.sigma_d = value
207
208         '''
209         Method returning the desired sigma.
210         '''
211         def getDesired(self):
212             return self.sigma_d
213
214         '''
215         Method returning the task Jacobian.
216         '''
217         def getJacobian(self):
218             return self.J
219
220         '''
221         Method returning the task error (tilde sigma
222         ).

```

```

199 '''
200 def getError(self):
201     return self.err
202 '''
203     Method setting the gain matrix K.
204
205     Arguments:
206     K(Numpy array): gain matrix
207 '''
208 def setGain(self, K):
209     self.K = K
210 '''
211     Method returning the gain matrix K.
212 '''
213 def getGain(self):
214     return self.K
215 '''
216     Method setting the feedforward velocity.
217
218     Arguments:
219     value(Numpy array): value of the feedforward
220     velocity
221 '''
222 def SetFeedforwardVelocity(self, value):
223     self.v_ff = value
224 '''
225     Method returning the feedforward velocity.
226 '''
227 def getFeedforwardVelocity(self):
228     return self.v_ff
229 '''
230     Subclass of Task, representing the 2D position
231     task.
232 '''
233 class Position2D(Task):
234     def __init__(self, name, desired, link=3,
235                 v_ff = np.zeros((2,1)), K = np.eye(2)):
236         super().__init__(name, desired, link, v_ff,
237                         K)
238         self.J = np.zeros((2,3)) # Initialize
239         self.err = np.zeros((2,1)) # Initialize
240         self.error vector
241
242     def update(self, robot):
243         # self.J = robot.getEEJacobian()
244         # self.err = self.sigma_d - robot.get ()
245         [0:2,3].reshape(2,1)
246
247         self.J = robot.getLinkJacobian(self.link)
248         [0:2, :] # Update task Jacobian
249         desired = self.getDesired().reshape(2,1)
250         expected = robot.getLinkTransform(self.link)
251         [0:2,-1].reshape(2,1)
252         self.err = desired - expected # Update task
253         error
254
255     '''
256     Subclass of Task, representing the 2D
257     orientation task.
258 '''
259 class Orientation2D(Task):
260     def __init__(self, name, desired, link=3, v_ff
261                 = np.zeros((1,1)), K = np.eye(1)):
262         super().__init__(name, desired, link, v_ff,
263                         K)
264         self.J = np.zeros((1,3))
265         self.err = np.zeros((1,1)) # Initialize with
266         proper dimensions
267
268     def update(self, robot):
269         self.J = robot.getOrientationJacobian(self.
270             link).reshape(1,3) # Update task
271             Jacobian
272         T1 = robot.getLinkTransform(self.link)[1,0]
273         T0 = robot.getLinkTransform(self.link)[0,0]
274
275         sigma = np.arctan2(T1,T0).reshape(1,1)
276         self.err = np.array(self.sigma_d - sigma ).
277             reshape(1,1)# Update task error
278
279     '''
280     Subclass of Task, representing the 2D
281     configuration task.
282 '''
283 class Configuration2D(Task):
284     def __init__(self, name, desired, link=3, v_ff
285                 = np.zeros((3,1)), K = np.eye(3)):
286         super().__init__(name, desired, link, v_ff,
287                         K)
288         self.J = np.zeros((3,3))
289         self.err = np.zeros((1,1)) # Initialize with
290         proper dimensions
291
292     def update(self, robot):
293         self.J = robot.getConfigJacobian(self.
294             link).reshape(3,3)# Update task
295             Jacobian
296         T1 = robot.getLinkTransform(self.link)[1,0]
297         T0 = robot.getLinkTransform(self.link)[0,0]
298         sigma_or= np.arctan2(T1,T0).reshape(1,1)
299         sigma_pos =robot.getLinkTransform(self.link)
300         [0:2,-1].reshape(2,1)
301
302         sigma = np.block([[sigma_pos], [sigma_or]]).
303             reshape(3,1)
304         self.err = self.sigma_d - sigma # Update
305         task error
306
307     '''
308     Subclass of Task, representing the joint
309     position task.
310 '''
311 class JointPosition(Task):
312     def __init__(self, name, desired, link=1, v_ff=
313                 None, K=None):
314         """
315         Constructor for JointPosition Task.
316
317         Arguments:
318         name (string): Name of the task
319         desired (numpy array): Desired joint
320             position
321         link (integer): Index of the joint (default
322             is 1)
323         v_ff (numpy array): Feedforward velocity
324             vector (default is 0)
325         K (numpy array): Gain matrix (default is
326             identity)
327         """
328         # Ensure correct dimensions for feedforward
329         velocity and gain matrix
330         if v_ff is None:
331             v_ff = np.zeros((1,1)) # Default
332             feedforward velocity (1x1)
333         if K is None:
334             K = np.eye(1) # Default gain matrix (1
335             x1)
336
337         super().__init__(name, desired, link, v_ff,
338                         K) # Call parent constructor
339
340         self.J = np.zeros((1, 3)) # Initialize
341         Jacobian (1x3)
342         self.err = np.zeros((1, 1)) # Initialize
343         error (1x1)
344
345     def update(self, robot):
346         """
347         Update method to compute Jacobian and error
348         for joint position control.
349
350         Arguments:
351         robot (Manipulator): The manipulator object
352         """

```

```

314 # Set Jacobian based on which joint is being controlled
315 self.J = np.zeros((1, 3)) # Reset Jacobian
316 if self.link < 3: # Ensure within valid joint range
317     self.J[0, self.link - 1] = 1 # Assign 1 to the correct joint
318
319 # Compute error between desired and current joint position
320 self.err = self.sigma_d - robot.getJointPos(self.link - 1)

```

3.3 lab4 exercise12.py

Below is an implementation of lab4 exercise12.py:

```

1 from lab4_robotics import * # Includes numpy import
2 import matplotlib.pyplot as plt
3 import matplotlib.animation as anim
4
5 # Robot model - 3-link manipulator
6 d = np.zeros(3) # Displacement along Z-axis
7 theta = np.array([np.pi/12, np.pi/6, np.pi/3]).reshape(3,1) # Rotation around Z-axis
8 alpha = np.zeros(3) # Rotation around X-axis
9 a = np.array([0.8, 0.6, 0.4]) # Displacement along X-axis
10 revolute = [True, True, True] # Flags specifying the type of joints
11 robot = Manipulator(d, theta, a, alpha, revolute) # Manipulator object
12 dt = 1.0 / 60.0 # Simulation time step
13
14 # Drawing preparation
15 fig = plt.figure()
16 ax = fig.add_subplot(111, autoscale_on=False, xlim=(-2, 2), ylim=(-2, 2))
17 ax.set_title('Simulation')
18 ax.set_aspect('equal')
19 ax.grid()
20 ax.set_xlabel('x [m]')
21 ax.set_ylabel('y [m]')
22 line, = ax.plot([], [], 'o-', lw=2) # Robot structure
23 path, = ax.plot([], [], 'c-', lw=1) # End-effector path
24 point, = ax.plot([], [], 'rx') # Target
25 PPx = []
26 PPy = []
27
28 # Task hierarchy definition
29 sigma_endeff = np.zeros((2,1)) # Desired end-effector position
30 sigma_config = np.block([[sigma_endeff], [0]]).reshape(3,1) # Desired end-effector configuration
31
32 # Exercise 1: Uncomment the appropriate task hierarchy (a, b, c or d)
33 #####
34 # Task hierarchies (lists of tasks): uncomment one
35
36 #a. One task -> 1: end-effector position
37 #tasks = [Position2D("End-effector position", np.array([1, 1]).reshape(2,1), 3)]
38
39
40 #b. One task -> 1: end-effector configuration
41 #tasks = [Configuration2D("End-effector configuration", sigma_config.reshape(3,1), 3)]
42
43
44 #c. Two tasks -> 1: end-effector position, 2: end-effector orientation
45 #tasks = [
46     Position2D("End-effector position", np.array([1,1]).reshape(2,1), 3),
47     Orientation2D("End-effector orientation", np.array([0]).reshape(1,1), 3)]
48
49 #d. Two tasks -> 1: end-effector position, 2: joint 1 position
50 #tasks = [
51     Position2D("End-effector position", np.array([1,1]).reshape(2,1), link=3),
52     JointPosition("Joint 1 position", np.array([0.0]).reshape(1,1), link=1)] # Joint 1
53
54 #####
55
56
57
58 # Exercise 2: Define tasks with tracking and gain matrices
59 tasks = [
60     Position2D("End-effector position", np.array([1,1]).reshape(2,1), link=3, v_ff=np.array([[0.1], [0.1]]), K=np.diag([2,2])),
61     Orientation2D("End-effector orientation", np.array([0]).reshape(1,1), link=2, v_ff=np.array([[0.05]]), K=np.array([[3]]))]
62
63 #####
64
65
66
67 error_vector = np.zeros((0, len(tasks))) # Error vector
68 err_counter = 0 # Error counter
69
70
71 # Simulation initialization
72 def init():
73     global tasks, sigma_endeff, error_vector
74     line.set_data([], [])
75     path.set_data([], [])
76     point.set_data([], [])
77     sigma_endeff = np.random.uniform(-1, 1, (2, 1)) # Random end-effector goal
78     sigma_config = np.block([[sigma_endeff], [np.pi / 12]]).reshape(3,1) # Desired end-effector configuration
79
80
81 for task in tasks:
82     if task.name == "End-effector position":
83         task.setDesired(sigma_endeff)
84         task.SetFeedforwardVelocity(np.array([[0.1], [0.1]])) # Tracking velocity
85     elif task.name == "End-effector configuration":
86         task.setDesired(sigma_config)
87
88
89 return line, path, point
90
91 # Simulation loop
92 def simulate(t):
93     global tasks, robot, PPx, PPy, err_counter, sigma_endeff, error_vector
94
95     ### Recursive Task-Priority algorithm ###
96     dof = robot.getDof() # Get robot degrees of freedom
97     P = np.eye(dof) # Initialize null-space projector
98     err_acc = np.zeros((1, 0)) # Initialize accumulated error
99     dq = np.zeros((dof, 1)) # Initialize joint velocity
100
101     #####

```

```

101 # Uncomment for Desired trajectory (Exercise 2
      tracking)
102
103 sigma_endeff = np.array([np.cos(t), np.sin(t)])
      reshape(2,1) # Circular trajectory
104 feed_vel = np.array([-np.sin(t), np.cos(t)]).
      reshape(2,1) # Corresponding velocity
105 sigma_config = np.block([[sigma_endeff], [np.pi
      /2]]).reshape(3,1) # Desired end-effector
      configuration
106 #####
107
108
109
110 for task in tasks: # Loop over tasks
111
112     #####
113     # Exercise 2: Enable tracking by setting
      dynamic desired values (uncomment for
      tracking)
114     if task.name == "End-effector position":
115         task.setDesired(sigma_endeff)
116         task.SetFeedforwardVelocity(feed_vel)
117     #####
118
119
120     task.update(robot) # Update task state
121     J = task.getJacobian() # Compute task
      Jacobian
122     Jbar = J @ P # Compute augmented Jacobian
123
124     err_acc = np.block([err_acc, np.linalg.norm(
      task.getError())]) # Accumulate task
      error
125     dq = dq + DLS(Jbar, 0.1) @ (task.getGain() @
      task.getError() - J @ dq) # Compute
      control update
126
127     P = P - PseudoInverseJacob(Jbar) @ Jbar #
      Update null-space projector
128
129 # Update robot state
130 robot.update(dq, dt)
131
132 # Store error
133 error_vector = np.block([[error_vector], [
      err_acc]])
134 err_counter += 1
135
136 # Update visualization
137 PP = robot.drawing()
138 line.set_data(PP[0, :], PP[1, :])
139 PPx.append(PP[0, -1])
140 PPy.append(PP[1, -1])
141 path.set_data(PPx, PPy)
142 point.set_data(sigma_endeff[0], sigma_endeff[1])
143
144 return line, path, point
145
146 # Run simulation
147 animation = anim.FuncAnimation(fig, simulate, np.
      arange(0, 10, dt),
148                               interval=10, blit=
      True, init_func=
      init, repeat=
      True)
149
150 plt.show()
151
152 # Plot the evolution of task errors
153 total_time = np.arange(0, err_counter * dt, dt)
154 fig = plt.figure()
155 ax2 = fig.add_subplot(111, autoscale_on=False, xlim
      =(0, err_counter * dt), ylim=(0, error_vector.
      max()))
156 ax2.set_title('Task-Priority -> {} Tasks'.format(len
      (tasks)))
157 ax2.set_xlabel('Time (s)')
158 ax2.set_ylabel('Error')
159 ax2.grid()
160
161 for i, task in enumerate(tasks):
162     ax2.plot(total_time, error_vector[:, i], label=
      task.name + " of link " + str(task.link))
163
164 ax2.legend()
165 plt.show()

```

4 RESULTS AND DISCUSSION

The result of lab is obtained as:

4.1 Exercise 1 (Recursive Task Priority)

Hierarchy Type	Controls	Use
a. One Task: End-Effector Position	(x, y) only	When orientation is not important.
b. One Task: End-Effector Configuration	(x, y, θ)	When both position and orientation need control.
c. Two Tasks: Position + Orientation	(x, y) first, then θ	Ensures orientation does not interfere with position tracking.
d. Two Tasks: Position + Joint 1 Position	$(x, y) +$ Joint Constraint	Used when a joint must be fixed while reaching the target.

TABLE 2: Task Hierarchies and Their Applications

a. One task 1: end effector position

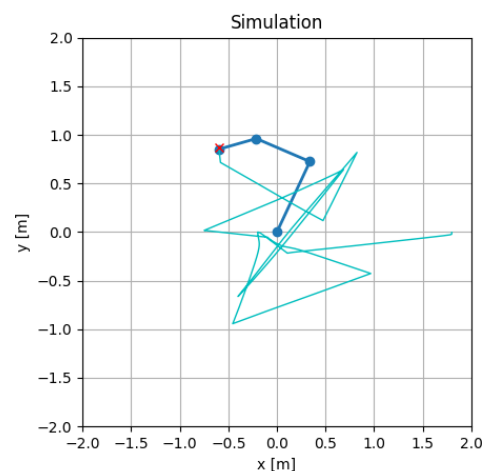


Fig. 2: a. One task 1: end effector position simulation

- The simulation shows the end effector's movement towards a target position in a 2D plane, with the path traced by the blue line and the red cross marking the target.

- The graph plots the error in the end effectors position over time, showing periodic oscillations as the system corrects its position.

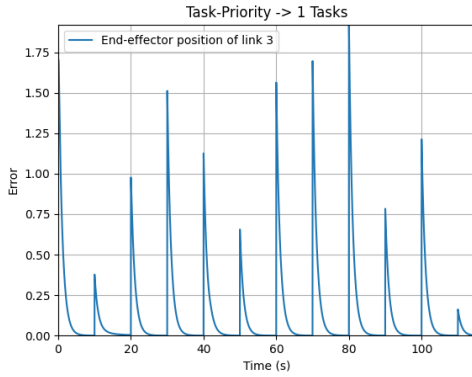


Fig. 3: a. One task 1: end effector position error

b. One task 1: end-effector configuration

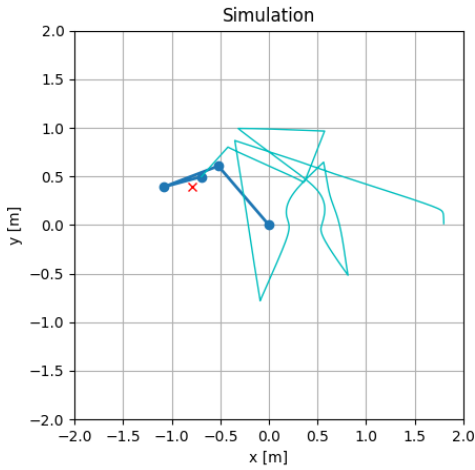


Fig. 4: b. One task 1: end effector configuration simulation

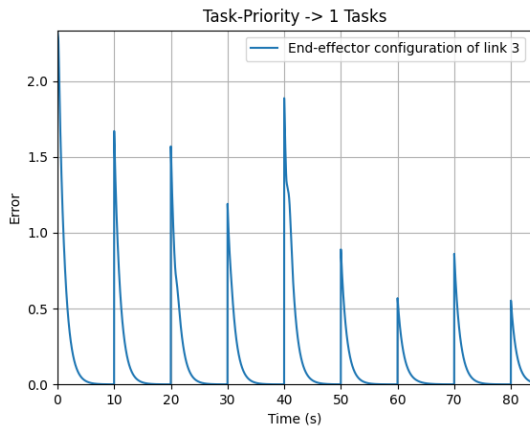


Fig. 5: b. One task 1: end effector configuration error

c. Two task1: end-effector position, 2: end-effector orientation

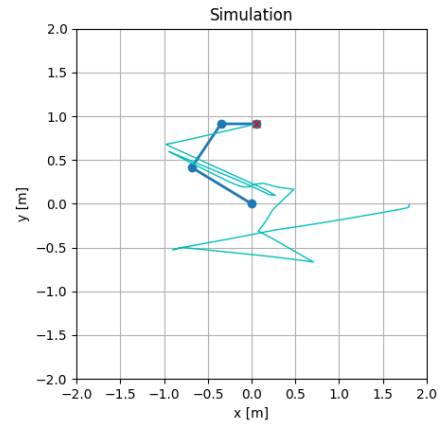


Fig. 6: c. Two tasks 1: end-effector position, 2: end-effector orientation simulation

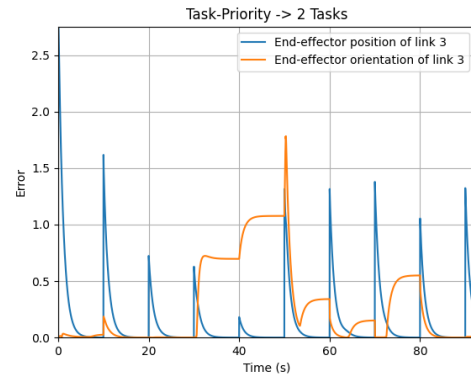


Fig. 7: c. Two tasks (1: end-eff position, 2: end-eff orientation error)

d. Two tasks 1: end-effector-position, 2: joint1 position

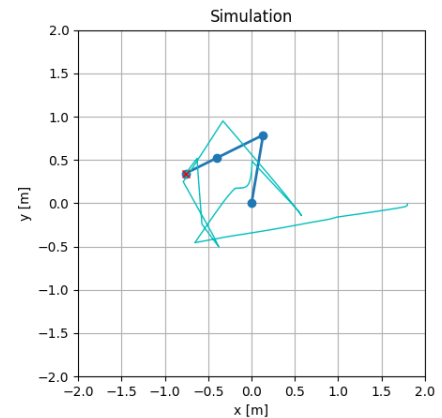


Fig. 8: d. Two tasks (1: end-eff position, 2: joint 1 position simulation)

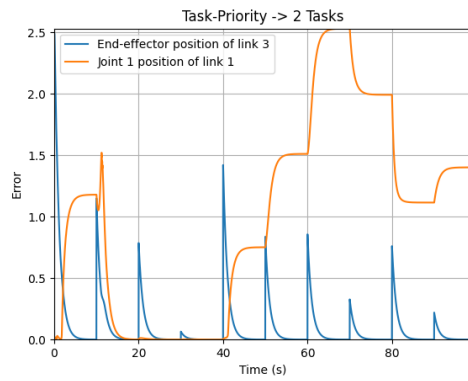


Fig. 9: d. Two tasks 1: end-effector position, 2: joint 1 position error

4.2 Exercise 2 (Updated recursive Task-Priority implementation)

The exercise 2 is implemented in two step: first without tracking and second with tracking.

4.2.1 1) position of the end-effector, 2) orientation of the second link equal 0. (Without Tracking)

Gain $K=1$ (without Tracking)

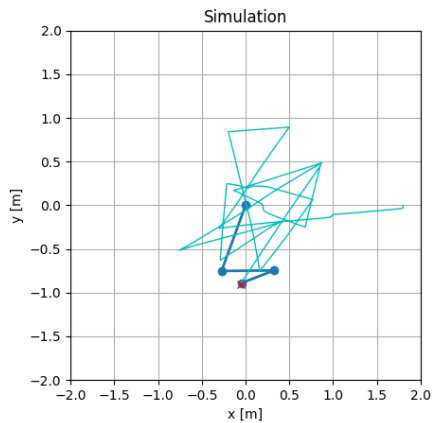


Fig. 10: Simulation for $K=1$

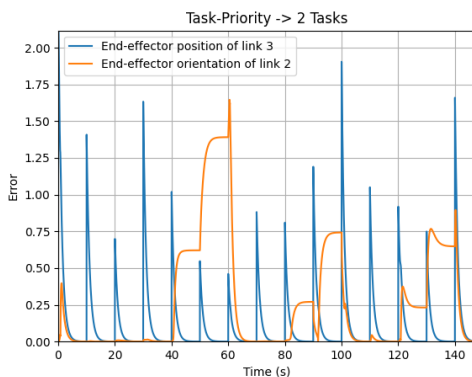


Fig. 11: error for $K=1$

Gain $K=10$ (without Tracking)

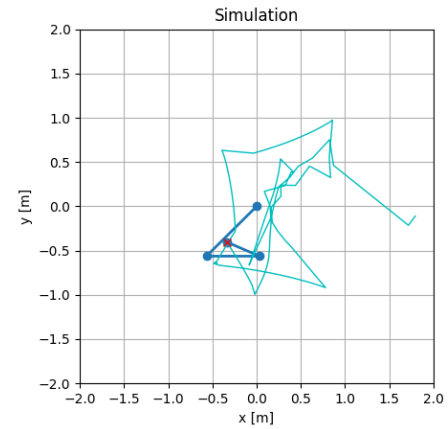


Fig. 12: Simulation for $K=10$

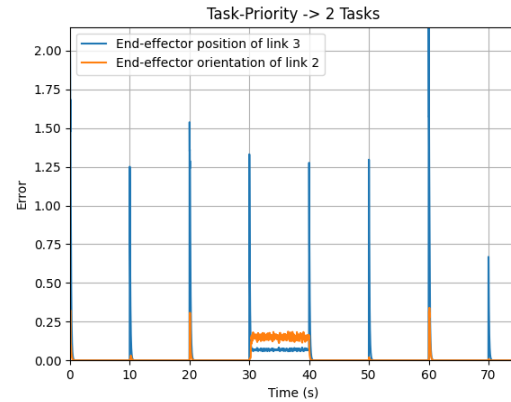


Fig. 13: error for $K=10$

Gain $K=100$ (without Tracking)

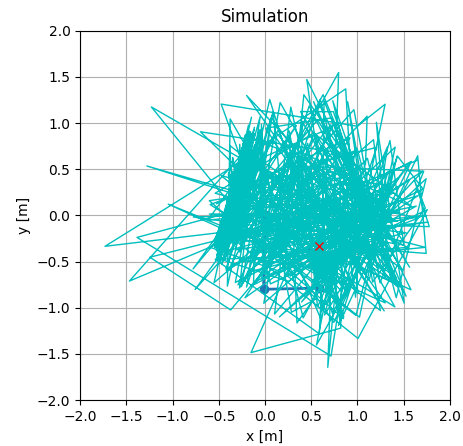


Fig. 14: Simulation for $K=100$

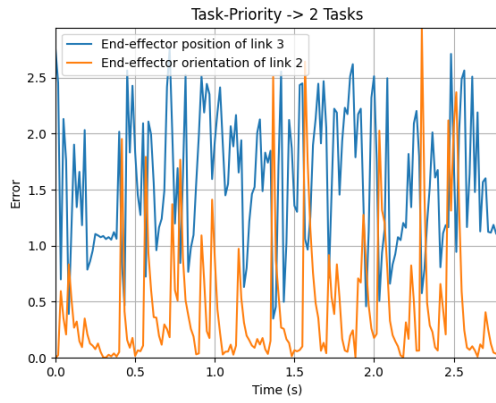


Fig. 15: error for K=100

4.2.2 1) position of end effector, 2) orientation of the 2nd link equal 0.

With Tracking In this part the end effector tracks the goal point

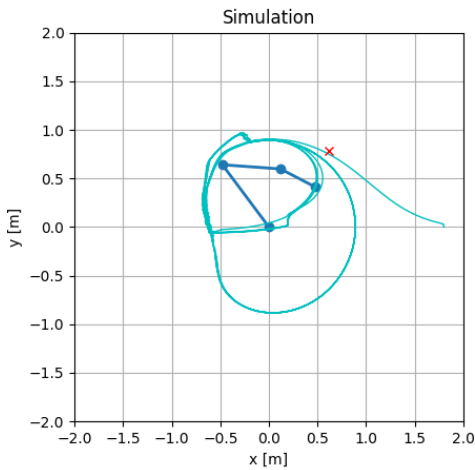


Fig. 16: With Tracking simulation

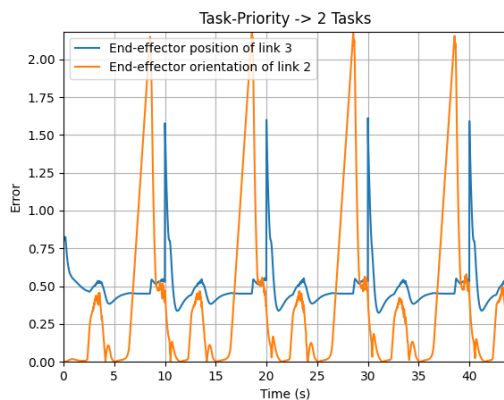


Fig. 17: With Tracking Error

5 CONCLUSION

The implementation of the Task-Priority control algorithm for a 3-link planar manipulator successfully demonstrated hierarchical task management in robotic systems. Exercise 1 confirmed that the manipulator could efficiently perform multiple tasks in a prioritized order, ensuring the execution of higher-priority tasks while accommodating lower-priority ones in the null space. Exercise 2 extended this functionality by incorporating dynamic trajectory tracking, gain matrices for better control, and feedforward velocity components. The simulation results showed that the manipulator could effectively adapt to different task hierarchies and achieve desired positions and orientations, while the error plots provided insights into the task error evolution over time.